WEEK 05 · RAZOR PAGES FORM FLOW

讓表單
真的收到資料

小安的活動網站已經好看了，但同學按下「送出」後，資料要去哪裡？今天把畫面接上 ASP.NET 後端，完成第一條看得見的後端資料路線。

下載學生任務包 ↓　下載 PDF ↓



EXPECTED RESULT · 先看本週終點

今天先證明一件事：畫面輸入真的進入 C#。



ASP.NET 回應

報名資料已收到

參加者：小安

Email：student@example.com

場次：週三晚上

本週界線：先做到「收到與回應」；資料規則留到第六週。

開啟第五週成功路線 Demo ↗

MISSION MAP

三節課，只完成「收到資料」這一條路。

1

建立與認路

建立 Razor Pages、跑出 localhost，只認畫面、C#、CSS。

2

做出 POST 表單

理解 form、label、asp-for，完成可操作的報名表。

3

讓後端收到

用 Input、BindProperty 與 OnPost，顯示成功摘要。

第五週不教驗證規則；第六週再處理「收到不等於接受」。

THE WHOLE JOURNEY

本週追蹤一筆資料經過五個站。



下一站　從頭看

使用者先在瀏覽器填入課堂假資料。

這五站會貫穿第五週

ZERO TO ONE · PROJECT START

先不要急著貼表單。
第一個完成點是：自己的 Razor Pages 專案能跑。

1 建立
Razor Pages 專案

→

2 執行
開啟 localhost

→

3 認路
畫面、C#、CSS

→

4 完成
表單、POST、收到

主要路線

使用 Visual Studio 2026 建立 **ASP.NET Core Web App (Razor Pages)**。

卡住時的備援

用 `dotnet new webapp` 建立同一種專案，不換框架、不重做整頁。

開啟從零開始指南 ↗

START 01 · VISUAL STUDIO

建立一個乾淨的 Razor Pages 專案。

1

選範本

「建立新的專案」→ 搜尋 Razor → 選 ASP.NET Core Web App (Razor Pages)。

不要選 MVC、Web API、Blazor 或 Empty。

2

填設定

專案名稱：`Week05Registration`
Framework：.NET 10.0 (LTS)
Authentication：None

保留 HTTPS；其餘先用預設值。

建立位置要記得：請放在自己找得到、可寫入的資料夾；不要直接放進 ZIP 或雲端唯讀預覽。

範本名稱與框架選對，後面才會看到 Pages 資料夾

START 02 · TERMINAL FALLBACK

Visual Studio 找不到範本時，終端機也能建立同一種專案。

```
dotnet --version
dotnet new webapp -n Week05Registration -f net10.0
cd Week05Registration
dotnet watch
```

每一行在做什麼

`--version`：確認 .NET SDK 可用。

`new webapp`：建立 Razor Pages 專案。

`cd`：進入專案資料夾。

`watch`：啟動並在修改後自動重建。

這是備援路線：Visual Studio 已成功建立者不必重做。

若 --version 失敗，先回第二週補安裝，不要繼續貼程式

暫停講解 · 學生實作 0

從空白到第一次執行

25 分鐘

學生現在完成

1. 建立 `Week05Registration` Razor Pages 專案。
2. 按 `Ctrl + F5`；或執行 `dotnet watch`。
3. 在瀏覽器打開終端機 / Visual Studio 顯示的 localhost 網址。

完成條件

- 首頁能顯示，且網址以 `http://localhost` 或 `https://localhost` 開頭。
- 停止後能再啟動一次。
- 能在方案總管找到 `Pages` 與 `wwwroot`。

老師巡視：範本是否為 Razor Pages、Framework 是否為 .NET 10。

6 分鐘提醒：還沒看到 localhost 的先舉手，不先貼表單。

快 / 慢分流：快者找 Index 兩檔；慢者先只求首頁成功。

逐步指南 ↗

時間到先看 localhost；未成功者結伴排查建立與啟動步驟

PROJECT MAP · 先認路，不用背

今天只要知道：畫面、C#、CSS 在哪裡。

```
Week05Registration/
├─ Pages/
│  ├─ Index.cshtml
│  └─ Index.cshtml.cs
└─ wwwroot/
   └─ css/site.css
```

今天只在三處移動

改畫面：`Pages/*.cshtml`

處理資料：`Pages/*.cshtml.cs`

改樣式：`wwwroot/css/`

先認路：`Index.cshtml` 與 `Index.cshtml.cs` 是同一個 Razor Page 的合作夥伴。其他檔案先知道存在，未來用到再介紹。

POINT 01 · GET VS POST

GET 是拿報名頁，
POST 是交出欄位資料。

第一次開啟首頁，瀏覽器向伺服器拿 HTML；填完按下送出，才把 form 內的資料交給後端。

GET / 拿頁面

```
瀏覽器 ── GET / ──→ 伺服器
瀏覽器 ←── 表單 HTML ──
```

對應 Razor Pages 的 **OnGet**。

POST / 交資料

```
瀏覽器 ── POST / ─→ 伺服器
          名稱、Email、場次
```

對應今天要完成的 **OnPost**。

POINT 02 · FORM BOUNDARY

form 像一個資料信封，只有信封裡的欄位會一起送出。

```
<form method="post">
  <label>顯示名稱</label>
  <input ...>

  <button type="submit">
    送出報名
  </button>
</form>
```

先預測

如果 input 放在 </form> 後面，按下送出時，它會一起送嗎？

```
</form>
<input name="note"> <!-- Form 表單外 -->
```

不會

會

先選答案，再用「信封範圍」解釋。

method="post" 描述這次送出的方式

POINT 03 · LABEL BEFORE STYLE

label 不是裝飾，
它把「問題」和「輸入位置」連起來。




```
<label for="nameDemo">
  顯示名稱
</label>
<input id="nameDemo">
```

Placeholder 只是範例，輸入後會消失；不能取代 label。

POINT 04 · FIELD NAME

後端屬性還沒建立前，
先用 name 標出資料的送件地址。

```
Pages/
├─ Index.cshtml       ← 先完
└─ Index.cshtml.cs    ← 下一
```

```
<label for="displayName">顯示名稱</label>
<input id="displayName"
       name="Input.DisplayName">
```

label

for → id

＋

送出名稱

name → 後端地址

第三節建立 Input 後，再把這兩行改成 asp-for。

每個停點都必須能編譯；不先引用不存在的屬性

暫停講解 · 學生實作

PAUSE & PRACTICE 01

25 分鐘

先完成一份可編譯、可操作的表單。

查看本週成功路線 Demo ↗

學生現在完成

1. 使用 `Index-form-week05-starter.txt`。
2. 加入 form、名稱、Email、場次及備註。
3. 每個 label 的 for 對到 input 的 id。
4. 以 `name="Input.*"` 預留後端地址。

完成條件

- 專案仍可編譯並顯示表單。
- Tab 能依序進入每個欄位。
- 點 label 會對焦正確欄位。
- 390px 沒有整頁水平捲動。

老師巡視：不要把完整 html/head/body 貼進 Razor 首頁。

第 12 分鐘：還沒完成者先只保留名稱、Email、場次三欄。

快學生：改善 label 與輸入框間距，不新增功能。

☐ 我已在自己的 localhost 編譯並操作表單。

時間到：抽 2 位同學指出 form 範圍、for / id 與 name

CHECKPOINT 01

表單階段也必須保持可編譯。

1

Razor 頂端

`@page`、`@model` 是否仍存在？不要貼第二組 html/head/body。

2

form 邊界

欄位與 submit 按鈕是否都包在同一個 form 裡？

3

欄位地址

`for` 對到 `id`；`name` 先寫成 `Input.屬性`。

此時按下送出只會重新載入、沒有摘要，是合理的。但 Build error 不是完成狀態；下一輪才建立 Input 與 OnPost。

POINT 05 · PAGE MODEL

Index.cshtml 負責呈現，Index.cshtml.cs 負責處理。

同一個 Razor Page 有兩個合作檔案：一個面對使用者，一個接住請求與資料。

```
Pages/
├─ Index.cshtml
│   └─ 表單、欄位、結果
│
└─ Index.cshtml.cs
    └─ Input、BindProperty、OnGet、OnPost
```

POINT 06 · INPUT MODEL

四個欄位，不要散成四條線；用一個 Input 物件收在一起。

```
public class RegistrationInput
{
    public string DisplayName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Session { get; set; } = string.Empty;
    public string? Note { get; set; }
}
```

Input

DisplayName

Input

Email

Input

Session

Input

Note

同一個用途的輸入，組成一個模型

POINT 07 · MODEL BINDING

[BindProperty] 讓 ASP.NET 把 POST 欄位放進 Input。

```
[BindProperty]
public RegistrationInput Input { get; set; } = new();
```

Binding 做了什麼？

1. 找出送來的欄位名稱。
2. 比對 `Input.DisplayName` 等屬性。
3. 把文字轉成屬性需要的型別。
4. 把這次請求的值放進 Input 物件。

它不是資料庫。Binding 只把這次請求帶來的值放進 C# 物件。

POINT 08 · HANDLER

表單用 POST 送出時，Razor Pages 會尋找 OnPost()。

```
public bool Submitted { get; set; }

public void OnPost()
{
    Submitted = true;
}
```

和 OnGet 的分工

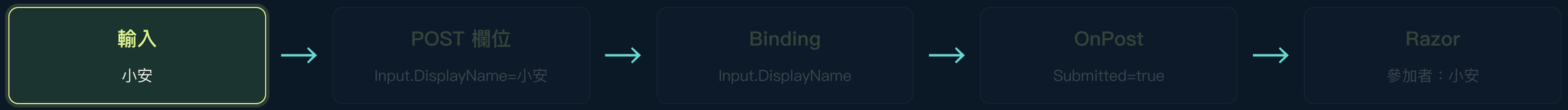
OnGet：第一次開啟頁面。

OnPost：使用者送出表單。

現在還沒有檢查內容。空白資料也會被當成成功；第三節才加上驗證閘門。

LIVE TRACE

「小安」如何走到成功摘要？



顯示下一步　重新追蹤

第一站：使用者在瀏覽器輸入「小安」。

每一站都可觀察，除錯才有方向

POINT 09 · RESPONSE

成功摘要不是另一個神祕頁面，是 Razor 根據狀態決定要不要顯示。

```
@if (Model.Submitted)
{
    <section class="alert alert-success">
      報名資料已收到
      參加者：@Model.Input.DisplayName
    </section>
}
```

現在能證明什麼？

後端收到這次 POST，並把 Input 的值放回 HTML 回應。

還不能證明什麼？

不能證明資料已存入 SQL Server，也不能證明重新啟動後還找得到。

正確描述系統現在做到哪裡

暫停講解 · 學生實作

PAUSE & PRACTICE 02

建立 Input，再把表單正式接上 PageModel。

25 分鐘

查看本週成功路線 Demo ↗

學生現在完成

1. 貼入 RegistrationInput、BindProperty、OnPost。
2. 把純 HTML 欄位改成 `asp-for="Input.*"`。
3. 加入 `Index-result-week05.txt` 的摘要。
4. 用兩組不同假資料送出。

完成條件

- 送出「小安」後，摘要也顯示「小安」。
- 換成另一個名稱，摘要跟著變。
- 中斷點能讀出 `Input.DisplayName`。
- 能說明目前沒有驗證與資料庫。

老師巡視：先看第一個錯誤與對應行，不讓 AI 重寫整頁。

第 12 分鐘：卡住者只保留名稱、Email、場次三欄。

快學生：改善成功摘要排版，不增加新欄位。

- [ ] 我已用兩組不同假資料完成 POST 與摘要驗證。

時間到：請一位同學沿五站路線解釋資料

CHECKPOINT 02

送出沒有結果時，沿資料路線查，不要直接重裝或重寫。

1 form
method="post" ？

→

2 asp-for
名稱一致？

→

3 BindProperty
Input 可接收？

→

4 OnPost
有執行？

→

5 Razor
條件成立？

> 除錯問題要具體：「我輸入____，按下____，預期____，實際看到____。」

現象越具體，修正範圍越小

MISSION COMPLETE · WEEK 05

你完成的不是一張靜態表單，而是第一條後端接收路線。

專案能重啟

localhost 可以停止後再次執行。

兩組資料都會變

輸入不同名稱，成功摘要跟著改變。

能說明五站

瀏覽器 → POST → Binding → OnPost → Razor。

下一週問題：空白名稱與 Email=abc 也可能被收到；收到不代表應該接受。

☐ 我能重新啟動專案 ☐ 表單可操作 ☐ 後端收到兩組不同資料